

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Sayed Afras S
Register Number	3122247001059
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	14.08.2026

1 Objective

The main aim of this experiment is to understand how ensemble learning can be used to get better and more stable classification results. Three approaches were worked with: Bagging, Boosting and Stacking. The models were trained on the Wisconsin Diagnostic Breast Cancer dataset and compared using accuracy, precision, recall, F1-score and ROC-AUC. Hyperparameters were also tuned using 5-fold cross-validation so that the final comparison was not based on just one random setting.

In simple terms, instead of depending on one model, the experiment checks what happens when several models are combined. This is useful in practical ML because a single model can make a few wrong decisions, while an ensemble can often reduce those mistakes.

2 Problem Statement

The task is to classify breast cancer samples into two classes: *malignant* and *benign*. The input contains 30 numerical measurements obtained from breast mass images. For every sample, the model has to predict the correct class.

The notebook first loads the dataset, checks its basic structure and class counts, and then performs an 80–20 train-test split. The numerical features are standard scaled. After that, Bagging, AdaBoost, Gradient Boosting and a Stacked Ensemble are trained. Grid search with stratified 5-fold cross-validation is used to select model settings.

The final goal is not only to get a high accuracy, but also to see which ensemble gives a good balance between precision, recall and F1-score. The confusion matrices and ROC curves are used as a practical check on the final predictions.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset
Dataset Source	Scikit-learn built-in breast cancer dataset (original dataset from UCI)
Number of Samples	569
Number of Features	30 numerical features
Number of Classes	2 – Malignant and Benign
Missing Values	No missing-value handling was required in the notebook
Train-Test Split	80% training and 20% testing

The dataset contains 569 samples and 30 numerical attributes. The target is encoded as 0 for malignant and 1 for benign in the notebook. There are 212 malignant samples and 357 benign samples. So the classes are not perfectly balanced, but both classes have enough samples for the stratified split used here.

4 Exploratory Data Analysis

The EDA in the notebook was kept fairly direct. First, the shape of the data and target counts were printed. A class distribution graph was then used to see whether one class was much larger than the other. A statistical summary was also generated for the first ten features, showing mean, standard deviation, minimum, median and maximum values.



Figure 1: Target class distribution in the breast cancer dataset

Inference: The graph shows 212 malignant samples and 357 benign samples. Benign is the larger class, so the dataset has some class imbalance. It is not extreme, but ignoring it could still affect the validation result. This is why the code uses a stratified split and StratifiedKFold, so both classes are represented properly in each split.

The statistical summary also shows that the 30 features are on very different scales. For example, mean area has values in the hundreds, while mean smoothness is much smaller. Feeding these values directly to models such as SVM can make the larger-scale features dominate, so scaling is useful before model training.

5 Data Preprocessing

The preprocessing part of the notebook is small but important. The data is divided into training and testing sets using an 80–20 split with `random.state=42`. Stratification is enabled using the target values.

After splitting, a `StandardScaler` is fitted only on the training data. The same scaler is then used to transform the test data. This avoids using test-set information while learning the scaling parameters.

Listing 1: Data splitting and feature scaling used in the notebook

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.20,  
3     random_state=42, stratify=y  
4 )  
5  
6 scaler = StandardScaler()  
7 X_train_scaled = scaler.fit_transform(X_train)  
8 X_test_scaled = scaler.transform(X_test)
```

The training set contains 455 samples and the test set contains 114 samples. Since the features are already numerical, no encoding step is needed. There is also no feature engineering step in this experiment. The main preprocessing choice is therefore standard scaling.

6 Machine Learning Algorithms

6.1 Bagging

Bagging, or Bootstrap Aggregation, trains multiple copies of a base model using different bootstrap samples. Their predictions are then combined. In this experiment, the base estimator is a Decision Tree.

The idea is quite practical: one Decision Tree may change a lot if the training data changes slightly. By training many trees on different samples and combining them, the overall model becomes less sensitive to that variation. This mainly helps in reducing variance.

The notebook searches over the number of estimators, the fraction of samples and the fraction of features. The searched values were 10, 30, 50 and 100 estimators; sample fractions of 0.5, 0.7 and 1.0; and feature fractions of 0.5, 0.7 and 1.0.

6.2 Boosting

Boosting builds models one after another. Each new model tries to focus more on the errors made by the earlier models. Two boosting models were tested: AdaBoost and Gradient Boosting.

AdaBoost was searched with 30, 50 and 100 estimators and learning rates of 0.01, 0.1 and 1.0. Gradient Boosting was searched with 30, 50 and 100 estimators, learning rates of 0.01, 0.1 and 0.2, and maximum tree depths of 3, 4 and 5.

Compared with Bagging, the models in boosting are not independent. The later models depend on the mistakes of earlier ones. This can reduce bias, but if the model becomes too focused on the training data, overfitting can also happen.

6.3 Stacked Ensemble

Stacking takes a slightly different approach. Instead of only averaging or voting between similar models, it combines different types of models and then lets another model learn how to combine their outputs.

Here the base learners are SVM, Gaussian Naive Bayes and a Decision Tree with maximum depth 4. Logistic Regression is used as the meta learner. The value of the Logistic Regression parameter C is searched over 0.01, 0.1, 1.0 and 10.0.

The best reported stacking configuration used Logistic Regression with $C = 10.0$. Its average cross-validation accuracy was 97.71% and its average F1 score was 0.9771.

7 Hyperparameter Selection

All ensemble models were tuned using `GridSearchCV`. A `StratifiedKfold` object with 5 folds, shuffling and random state 42 was used. Both accuracy and F1 were calculated during the search, while F1 was used for refitting the final model.

Model	Search Space
Bagging	$n_estimators = \{10, 30, 50, 100\}$; $max_samples = \{0.5, 0.7, 1.0\}$; $max_features = \{0.5, 0.7, 1.0\}$
AdaBoost	$n_estimators = \{30, 50, 100\}$; $learning_rate = \{0.01, 0.1, 1.0\}$
Gradient Boosting	$n_estimators = \{30, 50, 100\}$; $learning_rate = \{0.01, 0.1, 0.2\}$; $max_depth = \{3, 4, 5\}$
Stacked Ensemble	Meta learner: Logistic Regression; $C = \{0.01, 0.1, 1.0, 10.0\}$

A few rows printed by the notebook are shown below. They give a quick view of how changing the settings affects the cross-validation score.

Table 3: Bagging hyperparameter evaluation – rows displayed by the notebook

$n_estimators$	$max_samples$	Avg CV Accuracy (%)	Avg CV F1
10	0.5	95.16	0.9610
30	0.5	96.04	0.9683
50	0.5	96.04	0.9681
100	0.5	95.38	0.9630
10	0.7	94.95	0.9594

Table 4: Gradient Boosting hyperparameter evaluation – rows displayed by the notebook

$n_estimators$	$learning_rate$	Avg CV Accuracy (%)	Avg CV F1
30	0.01	92.97	0.9449
50	0.01	93.41	0.9481
100	0.01	93.85	0.9515
30	0.01	93.85	0.9513
50	0.01	93.19	0.9459

Table 5: Stacked ensemble cross-validation result

Base Models	Meta Learner	Accuracy (%)	F1
SVM, Naive Bayes, Decision Tree	Logistic Regression ($C = 10.0$)	97.71	0.9771

8 Results

The final models were evaluated on the held-out test set. The notebook reports the following values.

Table 6: Final test-set performance comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	ROC-AUC
Bagging	94.74	95.83	95.83	95.83	0.9926
AdaBoost	95.61	94.67	98.61	96.60	0.9818
Gradient Boosting	95.61	94.67	98.61	96.60	0.9927
Stacked Ensemble	97.37	98.59	97.22	97.90	0.9950

The Stacked Ensemble gives the highest test accuracy at 97.37%. It also has the highest precision and F1-score among the four final models. AdaBoost and Gradient Boosting have the same test-set accuracy, recall and F1 in the notebook, although their ROC-AUC values are quite different.

9 Result Visualization

9.1 Confusion Matrices

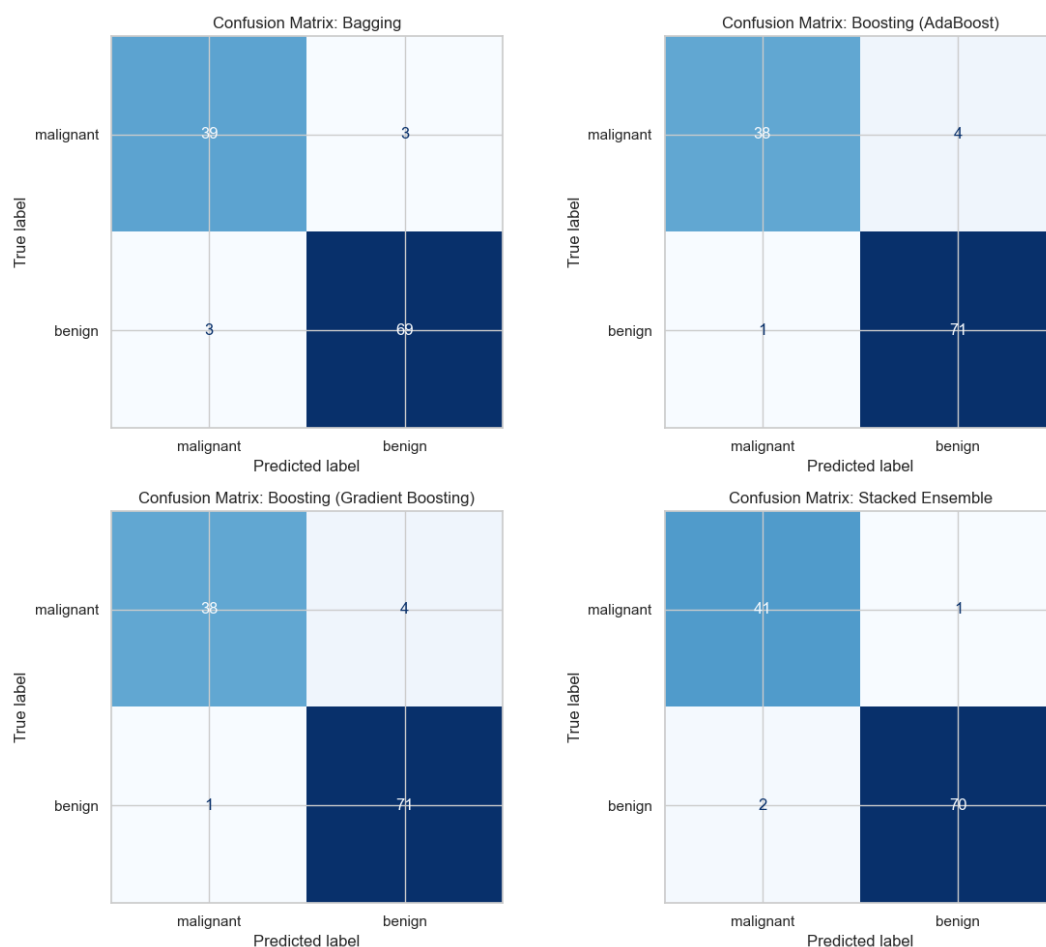


Figure 2: Confusion matrices for the four ensemble models

Inference: Bagging classified 39 malignant and 69 benign samples correctly, with 3 errors in each direction. AdaBoost and Gradient Boosting both classified 38 malignant and 71 benign samples correctly. The Stacked Ensemble had 41 correct malignant predictions and 70 correct benign predictions, with only three wrong predictions in total. So the stacked model gives the cleanest confusion matrix in this test.

9.2 ROC Curve

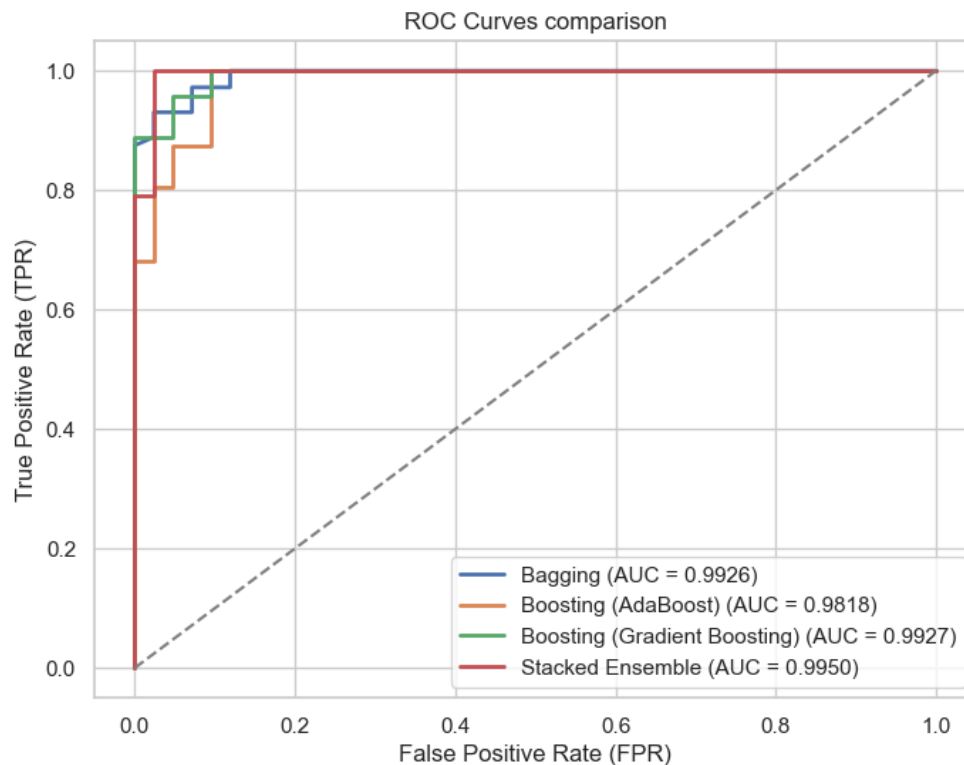


Figure 3: ROC curve comparison of the ensemble models

Inference: All four curves stay well above the random classifier line. The Stacked Ensemble has the highest AUC at 0.9950, which indicates very strong separation between the two classes. Bagging and Gradient Boosting are also close to it with AUC values of 0.9926 and 0.9927. AdaBoost has the lowest AUC of the four at 0.9818, but it still shows good classification performance.

10 Discussion

The results show a clear advantage for combining different models in this particular experiment. Bagging reached 94.74% test accuracy and produced balanced precision and recall values. Its confusion matrix is also fairly balanced, which fits the main purpose of Bagging: reducing the variation of a single tree.

AdaBoost and Gradient Boosting both reached 95.61% accuracy and 96.60% F1 on the test set. Their recall was 98.61%, which means they were good at finding the benign class used as the positive class in the dataset. However, the ROC-AUC of AdaBoost was lower than Gradient Boosting, so looking at only accuracy would miss a part of the model behaviour.

The Stacked Ensemble performed best overall. It reached 97.37% accuracy, 98.59% precision, 97.22% recall and 97.90% F1. Its ROC-AUC of 0.9950 was also the highest. The reason is not simply that it uses more models. It uses models with different learning behaviour: SVM creates a margin-based decision boundary, Naive Bayes works

from probabilistic assumptions, and the Decision Tree learns rule-like splits. Logistic Regression then learns how to combine these outputs.

One useful point from the experiment is that ensemble learning does not automatically mean every metric will increase. AdaBoost and Gradient Boosting have the same test metrics here, while their ROC-AUC values are different. Also, a model with a larger number of estimators is not always better. The cross-validation rows show small changes in score as the parameters are changed.

The experiment could be improved by reporting the complete hyperparameter result tables instead of only the first few rows, and by adding a precision-recall curve. More repeated train-test splits could also be used to check whether the same ranking of models stays stable. Still, for the given notebook, the Stacked Ensemble is the strongest choice based on the reported test results.

11 Observation Questions

1. How does Bagging reduce variance?

Bagging trains several base models on different bootstrap samples and combines their predictions. A single Decision Tree can change a lot when the training data changes slightly. Combining many trees reduces this variation, so the final prediction is generally more stable.

2. How does Boosting address model bias?

Boosting trains models sequentially and gives more attention to samples that were difficult for earlier models. Later models therefore try to correct earlier mistakes. This can reduce bias and make the final classifier stronger than a weak learner used alone.

3. Why does stacking benefit from heterogeneous models?

Different models make different kinds of errors. In this experiment, SVM, Naive Bayes and Decision Tree do not learn the data in the same way. The Logistic Regression meta learner can use their outputs together, instead of trusting only one type of model.

4. Which ensemble method performed best and why?

The Stacked Ensemble performed best on the final test set. It achieved 97.37% accuracy, 98.59% precision, 97.22% recall and 97.90% F1, along with an AUC of 0.9950. Its main advantage is the combination of three different base learners followed by a Logistic Regression meta learner.

12 Conclusion

In this experiment, Bagging, AdaBoost, Gradient Boosting and a Stacked Ensemble were implemented on the Wisconsin Diagnostic Breast Cancer dataset. The data was split into training and testing parts, standard scaled, and the ensemble models were tuned using stratified 5-fold cross-validation.

From the final results, the Stacked Ensemble gave the best overall performance. It obtained 97.37% test accuracy and the highest F1 and ROC-AUC among the models tested. The experiment also showed why ensemble methods are useful in practice: Bagging mainly improves stability, Boosting focuses on reducing errors, and Stacking combines the strengths of different models.

Overall, this experiment gave a practical view of ensemble learning rather than just the theory. It also showed that model comparison should use more than one metric, since accuracy alone does not always tell the full story.

13 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] Scikit-learn, “Ensemble methods documentation,” Online. Available: <https://scikit-learn.org/stable/modules/ensemble.html>
- [3] Scikit-learn, “BaggingClassifier documentation,” Online. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>
- [4] Scikit-learn, “Breast cancer dataset,” Online. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
- [5] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Dataset,” Online. Available: <https://archive.ics.uci.edu/>